CS 499 Milestone One Code Review Transcript

Slide 1 — Code Review

For my CS 499 code review, I'll be examining the Grazioso Salvare Animal Shelter Dashboard, which I originally developed in CS 340: Client/Server Development.

I'll be reviewing the original application, identifying opportunities for improvement, and discussing the enhancements I plan to implement throughout my capstone. My review will focus on three categories: Software Design and Engineering, Algorithms and Data Structures, and Databases.

Slide 2 — Project Overview

The Grazioso Salvare Animal Shelter Dashboard was developed to help an animal rescue organization identify dogs that meet the requirements for different types of rescue training. These include water rescue, mountain or wilderness rescue, and disaster or individual tracking.

The application uses Python and Dash to create an interactive interface that retrieves animal shelter records from MongoDB. The project is divided into two primary files. The first is the dashboard notebook, which contains the user interface, filtering logic, and visualizations. The second is a Python module containing the AnimalShelter class, which handles database operations.

The dashboard allows users to filter records according to specific rescue criteria, examine the results in an interactive table, and view breed distributions and geographic information. The database module supports the basic create, read, update, and delete operations.

Although the original application meets many of the functional requirements from CS 340, I identified several areas that could be improved. Some involve how the application is organized, while others relate to the efficiency of processing and retrieving animal records. There are also opportunities to strengthen data validation and security.

I selected this artifact because it provides an opportunity to demonstrate all three enhancement categories within one application. Rather than developing three unrelated projects, I can evaluate how improvements to the architecture, algorithms, and database operations affect the system as a whole.

Slide 3 — Existing Functionality: Dashboard Interface

I'll start with Software Design and Engineering by looking at how the existing dashboard is structured and what it allows users to do.

The Animal Shelter Dashboard uses Dash, a Python framework for building interactive web applications. One of the advantages of Dash is that the interface can be created using Python rather than developing a separate JavaScript frontend.

The code shown here defines the dashboard's layout using app.layout and html.Div. These components establish how the content is organized on the page, including the organization's logo, heading, and rescue selection controls.

The rescue options are created using dcc.RadioItems. Each option has a label displayed to the user and a corresponding value that the application uses to identify the selected rescue category.

The dashboard provides four options: Water Rescue, Mountain or Wilderness Rescue, Disaster or Individual Tracking, and Reset.

Reset is the default selection, which allows users to view the available animal records without applying one of the specific rescue filters.

When a user selects a different category, Dash passes that selection to a callback function. The function retrieves records that match the selected rescue criteria and updates the dashboard accordingly.

This allows shelter staff to examine animals suited for different rescue assignments without having to write MongoDB queries themselves.

Overall, the existing interface provides a straightforward way to interact with the shelter's animal records. The next slide shows how those records are displayed in the interactive data table.

Slide 4 — Existing Functionality: Interactive Data Table

Another important feature of the dashboard is the interactive data table, which displays animal shelter records retrieved from MongoDB.

The table is created using Dash's DataTable component. Before the records can be displayed, the application converts the information into a Pandas DataFrame and then into a format that Dash can use.

The code shown here defines several features that make the table interactive and easier to navigate.

For example, the page_size property is set to ten, so users see ten animal records at a time rather than having to scroll through the entire list.

The table also supports sorting and filtering. Users can organize records by different columns or narrow the displayed results based on specific information.

These features are enabled through the sort_action and filter_action properties, which are both set to native.

Another feature is single-row selection. This allows users to select an individual animal record, which the application can then use to update other components of the dashboard, including the geographic visualization.

The code also includes formatting options for column widths, text alignment, and header appearance to make the information easier to read.

Overall, the table provides shelter staff with an organized way to examine animal records and interact with the results of their rescue selections.

Now that I've covered the dashboard's existing functionality, I'll move into my code analysis and discuss the areas I identified for improvement.

Slide 5 — Code Analysis: Application Structure

Now that I've covered the dashboard's existing functionality, I'll move into my code analysis, starting with the application's structure.

One thing the original project does well is separate the database operations from the dashboard through the AnimalShelter class.

This class is defined in a separate Python module and contains methods for creating, reading, updating, and deleting animal records. This prevents the dashboard from having to repeat the same database operations throughout the notebook.

The constructor shown here accepts a username and password and uses MongoClient to establish a connection to MongoDB. It then identifies the AAC database and the animals collection that the application will use.

Although this provides some separation between the interface and database, I identified several limitations in the overall structure.

First, the database connection settings are defined directly within the code. This makes the application less flexible because changing the database environment would require modifying the source code.

Another concern is that much of the application logic remains inside the dashboard notebook. The callback functions handle user interactions, construct MongoDB queries, retrieve records, and process the results for display.

This means several responsibilities are combined rather than being handled by separate components.

As the application grows, this structure could make maintenance and troubleshooting more difficult. For example, changing how rescue criteria are evaluated could require modifying the same code responsible for updating the interface.

It also makes automated testing more complicated because the application logic is closely connected to the dashboard and database operations.

Overall, the existing CRUD module provides a useful starting point, but the application would benefit from a clearer separation of responsibilities.

The next slide examines a specific example of this issue within the dashboard's filtering callback.

Slide 6 — Code Analysis: Filtering Callback

The next area I examined was the dashboard's filtering callback, which determines which animal records are displayed when a user selects a rescue category.

The update_dashboard function uses a Dash callback to respond to changes in the filter-type component. Based on the user's selection, it constructs a MongoDB query to retrieve animals that meet the requirements for that rescue assignment.

For example, the Water Rescue filter looks for dogs belonging to specific breeds, including Labrador Retrievers, Chesapeake Bay Retrievers, and Newfoundlands.

The query uses a regular expression to match those breeds, and the i option makes the search case-insensitive. It also limits the results to intact females between 26 and 156 weeks old.

This approach works for the original project's requirements because the rescue criteria are already defined. However, I identified several limitations during my review.

The first is that the rescue requirements are hardcoded directly into the callback. If the organization changes its criteria or introduces another rescue category, the function would need to be modified.

Another concern is that the callback handles several responsibilities. It interprets the user's selection, constructs the database query, retrieves matching records, converts them into a DataFrame, and prepares the results for display.

Combining these responsibilities makes the function more difficult to maintain and test. For example, testing whether the rescue criteria are applied correctly would involve logic that is closely connected to the dashboard's interface.

The filtering process also depends on predefined conditions rather than a more flexible way of evaluating animal records.

Although the callback accomplishes its intended purpose, its structure makes it harder to expand the application's functionality without increasing the complexity of the code.

These findings, along with the architectural concerns from the previous slide, helped me identify the improvements I'll focus on for my Software Design and Engineering enhancement.

Slide 7 — Planned Enhancements: Software Design and Engineering

Based on the issues I identified during my code review, my first enhancement will focus on improving the application's structure and expanding its functionality.

The first change will be reorganizing the application into three separate layers: presentation, business logic, and data access.

The presentation layer will manage the dashboard and user interactions. The business logic layer will handle tasks such as validating information and applying rescue criteria, while the data access layer will remain responsible for communicating with MongoDB.

This addresses the issue I identified earlier, where the dashboard callbacks handle several responsibilities within the same function. Separating those responsibilities should make the application easier to maintain, troubleshoot, and test.

I also plan to introduce forms that allow authorized shelter staff to add and update animal records directly through the dashboard.

The existing create method shown here uses MongoDB's insert_one() operation to add a document to the collection. Although it performs a basic check for missing input, it doesn't verify whether all required fields are present or whether the information is valid.

For my enhancement, I'll introduce additional input validation to prevent incomplete or incorrect animal records from being submitted. I'll also implement authorization checks to restrict database modifications to appropriate users.

Another improvement will involve error handling. Rather than simply printing an error when an operation fails, the application should provide useful feedback without exposing unnecessary technical details.

Finally, I plan to develop automated tests to verify that the new functionality works correctly. These will include testing valid and invalid inputs, checking that unauthorized operations are rejected, and confirming that the application handles errors appropriately.

These enhancements will demonstrate my skills in software architecture, modular programming, validation, security, and automated testing.

They support course outcomes three, four, and five by evaluating the application's existing design, applying appropriate software development practices, and incorporating security into the enhancement process.

Overall, my goal is to make the dashboard more maintainable, reliable, and practical for shelter staff while establishing a stronger foundation for the remaining enhancements.

Slide 8 — Algorithms and Data Structures: Existing Filtering Logic

The second category of my code review is Algorithms and Data Structures. For this section, I examined how the dashboard processes rescue criteria to identify suitable animal records.

The existing application uses conditional statements to determine which MongoDB query should be executed based on the rescue category selected by the user.

The code shown here contains the filtering logic for Mountain or Wilderness Rescue and Disaster or Individual Tracking.

For Mountain Rescue, the application searches for specific breeds, including German Shepherds, Alaskan Malamutes, Siberian Huskies, and Rottweilers. It also limits the results to intact males between 26 and 156 weeks old.

The Disaster Tracking filter follows a similar approach but uses different requirements, including breeds such as Doberman Pinschers, Golden Retrievers, and Bloodhounds, with an age range of 20 to 300 weeks.

When the user selects Reset, the application uses an empty query to retrieve all available animal records without applying the rescue-specific restrictions.

This approach works for the original application because MongoDB handles the filtering, and the dashboard only needs to display the records that meet the selected requirements.

However, I identified a limitation in how the application evaluates potential rescue candidates.

The filtering logic determines whether an animal meets a set of predefined conditions, but it doesn't compare the animals that qualify.

For example, two dogs might both meet the requirements for Mountain Rescue, but the application doesn't determine whether one is a stronger candidate based on its individual characteristics.

The existing code also doesn't calculate suitability scores or rank the results. Every animal that meets the requirements is treated as an equally valid match.

This limits the amount of information available to shelter staff when comparing potential rescue candidates.

Although the filtering algorithm accomplishes its original purpose, there is an opportunity to improve how the application evaluates and prioritizes the matching records.

These findings provide the foundation for my Algorithms and Data Structures enhancement, which I'll explain on the next slide.

Slide 9 — Planned Enhancements: Algorithms and Data Structures

Based on the limitations I identified in the existing filtering logic, my second enhancement will focus on improving how the application evaluates and ranks potential rescue candidates.

The original dashboard can filter animals based on specific requirements and display information such as the most common breeds. The code shown here uses value_counts() and head(10) to identify the ten most common breeds for a visualization. However, this doesn't tell shelter staff which individual animals might be the strongest candidates for a particular rescue assignment.

To address this, I plan to develop a weighted scoring algorithm that evaluates animals based on their characteristics, including breed, age, and the selected rescue category.

Each characteristic will contribute to an overall suitability score based on its assigned weight. This will allow the application to compare animals and rank them rather than simply identifying whether they meet the basic requirements.

One of the main considerations for this enhancement is choosing an efficient way to identify the highest-ranking candidates.

A straightforward approach would be to calculate a score for every animal, sort the entire list, and select the highest-scoring results.

However, sorting the entire dataset may be unnecessary when shelter staff only need to see the top five or ten candidates.

For this reason, I plan to implement a min-heap priority queue. A min-heap allows the application to maintain a small group of the highest-scoring animals without having to sort every record.

As each animal is evaluated, its score will be compared against the lowest score currently stored in the heap. If the new animal has a higher score, it can replace the lowest-ranking candidate.

In terms of efficiency, sorting the complete list generally requires O(n log n) time, while maintaining the top k candidates with a min-heap requires approximately O(n log k) time.

This could provide a performance advantage when the database contains a large number of animals, but the user only needs a small number of recommendations.

However, I don't want to assume that using a more advanced data structure automatically makes the application better.

I'll compare the heap implementation against conventional sorting using datasets of different sizes. I'll measure execution time and memory usage, and verify that both approaches produce the same ranking results.

I'll also test how the algorithm handles missing information and animals with identical scores to ensure the results remain consistent.

These improvements will demonstrate my skills in algorithm design, data structure selection, computational complexity, and performance testing.

They primarily support course outcomes three and four by evaluating computational trade-offs and applying appropriate techniques to solve a practical problem. Presenting the results clearly will also support professional communication and help shelter staff make more informed decisions.

Overall, this enhancement will move the dashboard beyond basic filtering and provide a more useful way to identify potential rescue candidates for further evaluation.

Slide 10 — Databases: MongoDB Read Operations

The final category of my code review is Databases. For this section, I examined how the application retrieves, updates, and manages animal records in MongoDB.

The Animal Shelter Dashboard uses a separate CRUD module to communicate with the database. The code shown here contains the read() method, which is responsible for retrieving animal records.

The method accepts a query and uses MongoDB's find() operation to search the animals collection for matching documents.

It also checks whether a query was provided and includes exception handling in case the database operation fails.

When MongoDB executes the query, it returns a cursor, which allows the application to work through the matching records. However, the method immediately converts those results into a Python list before returning them to the dashboard.

This works for the original project, but I identified a potential performance issue.

Because the method converts the entire result into a list, every matching record is loaded into memory, even if the user only needs to view a small portion of the results.

The method also doesn't limit how many documents can be retrieved. For a small collection, this may not cause noticeable problems. However, as the number of animal records increases, retrieving the entire dataset could consume unnecessary memory and slow down the application.

Another limitation involves error handling.

If the database operation fails, the method prints an error message and returns an empty list. However, an empty list is also returned when a successful query doesn't find any matching records.

This means the dashboard cannot easily distinguish between a database error and a search that simply returned no results.

Overall, the read() method accomplishes its intended purpose, but the way it retrieves records and handles unsuccessful operations creates opportunities for improvement.

The next slide examines the existing update and delete methods, where I identified additional concerns involving data validation and integrity.

Slide 11 — Code Analysis: Database Updates and Validation

The next area I examined was how the CRUD module handles updating and deleting animal records.

The update() method uses MongoDB's update_many() operation to modify documents that match the supplied query. It uses the $set operator to change specific fields without replacing the entire document.

After the operation is completed, the method returns the number of documents that were modified.

The delete() method works similarly. It uses delete_many() to remove documents matching the query and returns the number of records that were deleted.

Both methods include basic input checks and exception handling. However, I identified several concerns related to validation and data integrity.

The first issue is that the methods check whether the supplied inputs are null, but they don't fully validate the information being provided.

For example, the update method doesn't verify whether the new values contain appropriate data types or whether required animal information remains valid.

Another concern is that both methods allow an empty query to be passed to MongoDB.

This is important because update_many() and delete_many() can affect multiple records at once. An empty query could potentially modify or delete every document in the collection rather than targeting a specific animal.

Although these methods provide the functionality required by the original project, unrestricted database operations introduce unnecessary risks, especially if they become accessible through the dashboard.

I also noticed that the methods primarily rely on printing error messages when something goes wrong. This provides limited information to the rest of the application about why an operation failed.

There are also no authorization checks within these methods to determine whether a user should be permitted to modify or delete records.

Overall, the update and delete methods accomplish their basic purpose, but the limited validation and unrestricted queries create potential security and data integrity concerns.

These findings, along with the performance limitations identified in the read method, form the basis of my planned database enhancements.

Slide 12 — Planned Enhancements: Database Security and Optimization

Based on the issues I identified during my database code review, my third enhancement will focus on improving how the application retrieves, manages, and protects animal records.

One of the first changes will involve database security. The original dashboard stores the MongoDB username and password directly in the source code, as shown here.

This creates a security concern because anyone with access to the source files could potentially obtain those credentials.

To address this, I plan to move the connection settings into environment variables so sensitive information isn't stored directly in the application's code.

Another improvement will focus on data retrieval. The original application uses db.read({}) to retrieve all animal records and then converts them into a Pandas DataFrame.

As I discussed earlier, this approach can become inefficient as the database grows.

For my enhancement, I'll change how the records are retrieved so the dashboard only requests the information needed for the page being viewed, rather than loading the entire collection into memory.

I also plan to introduce database indexes for fields that are frequently used together when searching animal records.

Indexes can help MongoDB find matching documents more efficiently. However, they also require additional storage and can affect the performance of database updates.

Because of this, I'll evaluate the existing queries before deciding which indexes would be most useful.

Another planned improvement involves MongoDB aggregation pipelines.

Currently, the application retrieves records and uses Pandas to calculate information such as breed frequencies.

With aggregation pipelines, MongoDB can perform operations such as filtering, grouping, and counting before returning the results to the application.

This could reduce the amount of information transferred between the database and the dashboard.

I'll also introduce database schema validation to help ensure animal records contain appropriate information and data types.

To address the risks I identified in the update and delete methods, I'll strengthen the validation of database operations and prevent unrestricted changes caused by empty queries.

Testing will be an important part of evaluating these enhancements. I plan to compare query execution times and use MongoDB's explain() method to examine how the database processes requests.

I'll also test whether invalid records and unsafe modification requests are rejected.

These enhancements will demonstrate my skills in database optimization, query analysis, data validation, security, and performance testing.

They support course outcomes three, four, and five by evaluating performance trade-offs, applying appropriate database techniques, and incorporating security into the application's design.

Overall, my goal is to make the database operations more efficient, reliable, and secure while supporting the additional functionality planned for the dashboard.

Slide 13 — Grazioso Salvare Dashboard Enhancement Roadmap

To summarize my code review, I identified opportunities to improve the Grazioso Salvare Animal Shelter Dashboard across all three enhancement categories.

For Software Design and Engineering, I'll focus on reorganizing the application into separate layers, adding staff management features, and strengthening validation, authorization, and automated testing.

For Algorithms and Data Structures, I'll develop a weighted scoring algorithm to rank potential rescue candidates. I'll also compare a min-heap priority queue with conventional sorting to determine which approach is more efficient.

For Databases, I'll improve how records are retrieved and processed, introduce appropriate indexes and aggregation pipelines, and strengthen database security and data validation.

Although these enhancements address different areas of the application, they're closely connected.

Improving the application's structure will make it easier to introduce new functionality without creating unnecessary dependencies. The ranking algorithm will provide a more useful way to evaluate animal records, while the database improvements will support more efficient and reliable data retrieval.

Together, these enhancements will demonstrate all five CS 499 course outcomes, including professional communication, collaboration and decision-making, evaluating computational trade-offs, applying appropriate computing techniques, and incorporating security into software design.

Throughout the enhancement process, I'll test the changes and compare their results against the original implementation to determine whether they provide measurable improvements.

My overall goal is to turn the original CS 340 project into a more maintainable, efficient, and secure application that better supports the needs of an animal rescue organization.

Slide 14 — Butters, My Rescue

I wanted to end by introducing Butters, my rescue dog. He's part of the reason I chose the Grazioso Salvare Animal Shelter Dashboard for my capstone.

Having a rescue of my own has given me a greater appreciation for the work these organizations do. This project allows me to apply my computer science skills to something I genuinely care about.